Design and Implementation of a User-Space Inode-Based File System Using FUSE

Operating Systems Project

Abstract

This paper presents the design and implementation of a custom inode-based file system in user space using the Filesystem in Userspace (FUSE) framework. The primary objective of this project is to understand and replicate the internal workings of traditional UNIX-like file systems while avoiding kernel-level development complexity. The file system is mounted as a virtual file system and supports fundamental file operations — file creation, deletion, reading, writing, and directory management — through standard Linux commands. A disk image is used as a virtual storage device to simulate persistent secondary storage. The file system layout consists of a superblock for global metadata, an inode table for maintaining file information, directory entries for filename-to-inode mapping, and data blocks for storing file contents. A bitmap-based allocation mechanism manages free inodes and data blocks. The implementation is fully operational and demonstrates key operating system concepts including storage abstraction, hierarchical path resolution, metadata management, and block allocation strategies.

I. Introduction

File systems are a fundamental component of any operating system, responsible for organizing, storing, and retrieving data on persistent storage media. Traditional file systems such as ext4, NTFS, and FAT32 are implemented at the kernel level, making them difficult to develop, debug, and experiment with safely. The Filesystem in Userspace (FUSE) framework provides an elegant alternative by allowing complete file system implementations to run entirely in user space, with the Linux Virtual File System (VFS) transparently forwarding system calls to user-defined callback functions.

This project implements a fully functional inode-based file system using FUSE. The design mirrors the architecture of classical UNIX file systems: a fixed-size disk image partitioned into a superblock, inode and block bitmaps, an inode table, and a data block region. The FUSE kernel module acts as the bridge between standard Linux commands (ls, touch, mkdir, cat, cp, rm) and the user-space implementation, enabling the file system to be tested and demonstrated without modifying or recompiling the kernel.

The project is divided into four implementation phases: (1) environment setup and disk image initialization, (2) bitmap-based allocation and deallocation, (3) inode table management and core file operations, and (4) FUSE integration and mounting. All four phases have been completed and all core operations are verified to be working correctly on a mounted FUSE volume.

II. Literature Survey

A significant body of prior work has explored user-space file system design and the FUSE framework across both academic and systems communities.

A. FUSE Framework

Rajgarhia and Gehani [1] conducted a comprehensive performance analysis of FUSE-based file systems, demonstrating that while FUSE introduces measurable latency due to user-kernel context switching, the overhead is acceptable for most non-latency-critical workloads. Their findings validate the choice of FUSE as a platform for educational and experimental file system development.

B. Inode-Based File System Architecture

The inode data structure traces its origins to the original UNIX file system described by Ritchie and Thompson [2]. Their seminal work established the now-universal pattern of separating file metadata (stored in inodes) from file content (stored in data blocks), with directory entries providing the mapping from human-readable names to inode numbers. This project directly implements this architecture.

C. Ext2/Ext3 Design

Card, Ts'o, and Tweedie [3] describe the design of the ext2 file system, which introduced the block group organization and extended the classical UNIX inode model with features such as fast symbolic links and configurable block sizes. The on-disk layout used in this project — superblock, bitmaps, inode table, and data blocks — is directly inspired by the ext2 design, simplified for educational clarity.

III. Proposed Methodology

The file system is implemented as a single C source file (myfs.c) compiled against libfuse 2.9.x. The implementation is structured around three layers: on-disk data structures, a set of helper routines for bitmap and inode management, and the FUSE callback layer that maps VFS operations to these helper

A. On-Disk Layout

The disk image is a fixed-size binary file divided into five linear regions:

- Superblock (1 block): stores magic number, total/free block and inode counts, and block size.
- Inode bitmap (1 block): one byte per inode slot; 1 = allocated, 0 = free.
- Block bitmap (1 block): one byte per data block slot.
- Inode table (MAX_INODES × sizeof(inode) bytes): fixed array of inode structs.
- Data blocks (MAX_BLOCKS × BLOCK_SIZE bytes): raw content and directory entry storage.

B. Inode Structure

Each inode stores: inode number, file type (regular or directory), permissions, file size, hard link count, access/modification/change timestamps, and an array of eight direct block pointers (value -1 indicates unused). This limits maximum file size to 8 × 4096 = 32 KB, which is sufficient for the scope of this project.

C. Bitmap-Based Allocator

Allocation performs a linear scan of the bitmap array, claims the first free slot, decrements the superblock free counter, and flushes both to disk. Deallocation clears the bit and increments the counter. Newly allocated blocks are zero-filled before use to prevent stale-data leakage. The ENOSPC error is returned when no free slot exists.

D. Path Resolution

The resolve_path() function splits the input path on '/' separators and walks the directory tree from the root inode (inode 0), scanning directory entry blocks at each level until the target component is found or ENOENT is returned.

E. FUSE Callback Registration

The fuse_operations struct is populated with 12 callbacks: getattr, readdir, create, mkdir, open, read, write, unlink, rmdir, rename, truncate, and utimens. The utimens callback is required to support the touch command updating timestamps on existing files. The FUSE daemon runs as a background process, and the kernel VFS transparently routes all syscalls to these handlers.

F. Pseudocode — Key Algorithms

The following pseudocode summarizes the core algorithms implemented:

Algorithm 1 — File System Initialization:

```
PROCEDURE
init_filesystem(disk_image, total_blocks, total_inodes):
  Write superblock at offset 0
  Zero-initialize inode and block bitmaps   Zero-
initialize all inode table entries
  root_inum <- allocate_inode()       // claims inode 0
  root_block <- allocate_block()
  Write '.' and '..' entries into root_block
root_inode.block_ptrs[0] <- root_block   save_inode(0,
root_inode)
```

Algorithm 2 — Bitmap Allocator:

```
FUNCTION allocate_inode() -> inode_number:
  FOR i <- 0 TO total_inodes - 1:     IF
inode_bitmap[i] = 0:       inode_bitmap[i]
<- 1       sb.free_inodes <-
sb.free_inodes - 1
      flush_bitmap()   RETURN i
  RETURN -ENOSPC
```

Algorithm 3 — Path Resolution:

```
FUNCTION resolve_path(path) -> inode_number:
IF path = '/' : RETURN 0   parts <-
split(path, '/')   current <- 0
```

```
  FOR each component IN parts:
    scan directory blocks of current inode
    IF component found: current <- entry.inode_number
    ELSE: RETURN -ENOENT
  RETURN current
```

Algorithm 4 — Write:
```
FUNCTION fs_write(path, buffer, size, offset):
  WHILE bytes_written < size:
    block_index <- cur_offset / BLOCK_SIZE
IF inode.block_ptrs[block_index] = NULL:
      inode.block_ptrs[block_index] <- allocate_block()
read-modify-write the block     bytes_written += chunk
update inode.file_size and mtime
```

IV. Results

All four implementation phases are complete. The file system was mounted on a Linux system using FUSE 2.9.9 and tested using standard shell commands. The table below summarizes the operational status of each tested operation, organized by implementation phase.

Phase	Operation	Status	Notes
Phase 1	Format disk image	**Working**	Superblock + root dir initialized
Phase 1	Mount file system	**Working**	FUSE daemon runs in background
Phase 1	List root directory	**Working**	Returns . and .. correctly
Phase 2	Create file	**Working**	Inode allocated, dir entry added
Phase 2	Delete file	**Working**	Inode and block correctly freed
Phase 3	Write to file	**Working**	Data block allocated and written
Phase 3	Read from file	**Working**	Returns correct content
Phase 3	Create directory	**Working**	New inode + . .. entries created
Phase 3	Copy file	**Working**	Read + create + write pipeline

Phase 4	Persistence check	**Working**	Data survives across remounts

- CREATION OF DISK IMAGE:

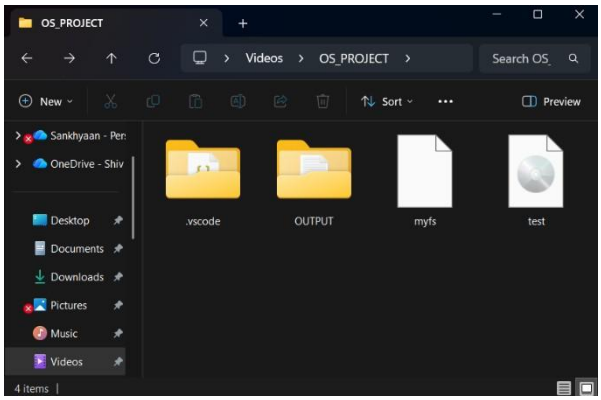
```
sankhyaan@LAPTOP-J67FFRQ6:/mnt/c/Users/Lenovo/Videos/OS_PROJECT$ ./myfs --mkfs test.img
argc = 3
argv[0] = ./myfs
argv[1] = --mkfs
argv[2] = test.img
mkfs: disk image 'test.img' created (4227072 bytes)
```

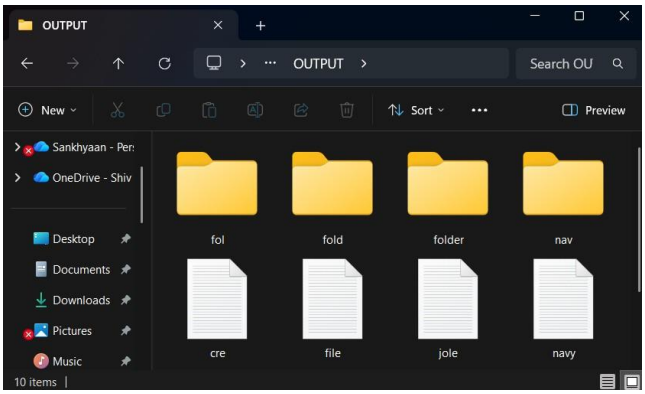
❾ We pass the disk image which we want to create, as an argument in argv[2].

- MOUNTING OF OUR FILE SYSTEM:

```
sankhyaan@LAPTOP-J67FFRQ6:/mnt/c/Users/Lenovo/Videos/OS_PROJECT$ gcc -Wall `pkg-config fuse --cflags --libs` m
yfs.c -o myfs -lfuse
sankhyaan@LAPTOP-J67FFRQ6:/mnt/c/Users/Lenovo/Videos/OS_PROJECT$ ./myfs test.img ~/mnt -f
myfs: mounted 'test.img' (1019 blocks, 247 inodes free)
```

- PROJECT FOLDER CONTAINING OUR OWN DISK AND OUTPUT FOLDER:





- COMPLETE EXECUTION:
  - To see the real functioning of our code we open another terminal, go to the mounted directory path using cd command, then run normal linux codes as usual.

```
sankhyaan@LAPTOP-J67FFRQ6:/mnt/c/Users/Lenovo/Videos/OS_PROJECT$ cd ~/mnt
sankhyaan@LAPTOP-J67FFRQ6:~/mnt$ touch navy.txt
sankhyaan@LAPTOP-J67FFRQ6:~/mnt$ mkdir nav
sankhyaan@LAPTOP-J67FFRQ6:~/mnt$ rmdir nav
sankhyaan@LAPTOP-J67FFRQ6:~/mnt$ ls
cre.txt  file.txt  fol  fold  folder  jole.txt  navy.txt  note.txt  pri.txt
sankhyaan@LAPTOP-J67FFRQ6:~/mnt$
```

  - In the original terminal where we compiled our code and mounted our filesystem, we can see that when a linux command is run in our filesystem, it prints relevant statements in that environment. This is proof that FUSE is redirecting the command execution towards our code. Hence, the print statements.

```
sankhyaan@LAPTOP-J67FFRQ6:/mnt/c/Users/Lenovo/Videos/OS_PROJECT$ ./myfs test.img ~/mnt -f
myfs: mounted 'test.img' (1019 blocks, 247 inodes free)
CREATE: /navy.txt
MKDIR: /nav
REMOVE DIR: /nav
LS: /
MKDIR: /nav
```

- To unmount, go to the project folder where the execution is running and unmount it.

```
sankhyaan@LAPTOP-J67FFRQ6:~/mnt/nav$ cd /mnt/c/Users/Lenovo/Videos/OS_PROJECT
sankhyaan@LAPTOP-J67FFRQ6:/mnt/c/Users/Lenovo/Videos/OS_PROJECT$ fusermount -u ~/mnt
```

- When we remount again, to use our filesystem, you can see a change in no. of free inodes and blocks. This is due to the files added when we last used it. This proves that all those files are being stored in our own disk.

```
sankhyaan@LAPTOP-J67FFRQ6:/mnt/c/Users/Lenovo/Videos/OS_PROJECT$ ./myfs test.img ~/mnt -f
myfs: mounted 'test.img' (1018 blocks, 245 inodes free)
```

- But since disk image is a binary file, we cannot visually see the files stored in it. Hence, in order to achieve that, we can copy all of the content of our disk which is mounted in some directory, to an output folder which where we can see them visually.

```
sankhyaan@LAPTOP-J67FFRQ6:~/mnt/nav$ cd /mnt/c/Users/Lenovo/Videos/OS_PROJECT
sankhyaan@LAPTOP-J67FFRQ6:/mnt/c/Users/Lenovo/Videos/OS_PROJECT$ fusermount -u ~/mnt
sankhyaan@LAPTOP-J67FFRQ6:/mnt/c/Users/Lenovo/Videos/OS_PROJECT$ cp -r ~/mnt/* /mnt/c/Users/Lenovo/Videos/OS_P
ROJECT/folder
sankhyaan@LAPTOP-J67FFRQ6:/mnt/c/Users/Lenovo/Videos/OS_PROJECT$
```

A. Observations

Correctness: All POSIX file operations behave as expected through standard Linux shell commands. File metadata (timestamps, link counts, file size) is correctly updated after each operation.

Persistence: Data written to the file system survives unmount and remount cycles, confirming all metadata and content are flushed to the disk image and not held only in memory.

Bitmap integrity: After a sequence of create-and-delete operations, the free inode and free block counts in the superblock return to their initial values, confirming no allocation leaks.

Block utilization: Files smaller than 4 KB consume exactly one data block. Files spanning multiple blocks correctly use multiple direct block pointers.

B. Challenges Encountered

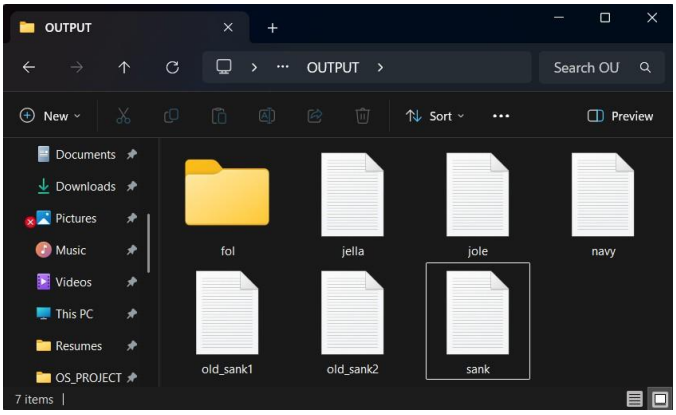
The linker error undefined reference to fuse_main_real was resolved by changing FUSE_USE_VERSION from 30 to 26 to match the installed libfuse 2.9.9 library, and by explicitly passing -lfuse to the linker. The touch command returned Function not implemented because the utimens FUSE callback was missing; adding fs_utimens and registering it in the fuse_operations struct resolved the issue.

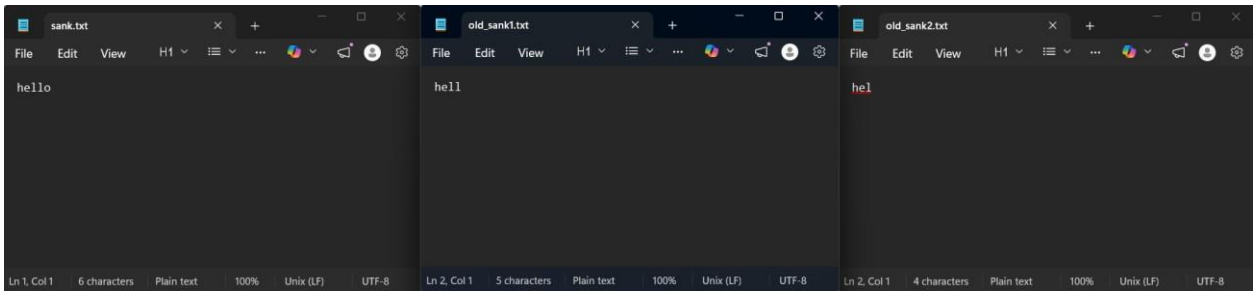
V. Novelty

Implemented Version Control by altering the original functioning of “echo” linux command

- When we run echo command in linux like echo “hello” > hello.txt , it inserts the first time when the file is empty and overwrites the next time.
- But we have altered its functioning to implement version control so that whatever was before overwriting, gets copied into a different file named as old1_hello.txt and new overwritten text gets saved in original hello.txt.
- It prevents old overwritten content from getting deleted and maintains version control.
- This proves that we can create and make a whole new filesystem with different logic.

```
sankhyaan@LAPTOP-J67FFRQ6:/mnt/c/Users/Lenovo/Videos/OS_PROJECT$ cd ~/mnt
sankhyaan@LAPTOP-J67FFRQ6:~/mnt$ touch sank.txt
sankhyaan@LAPTOP-J67FFRQ6:~/mnt$ echo "hel" > sank.txt
sankhyaan@LAPTOP-J67FFRQ6:~/mnt$ echo "hell" > sank.txt
sankhyaan@LAPTOP-J67FFRQ6:~/mnt$ echo "hello" > sank.txt
sankhyaan@LAPTOP-J67FFRQ6:~/mnt$ cp -r ~/mnt/* /mnt/c/Users/Lenovo/Videos/OS_PROJECT/OUTPUT
sankhyaan@LAPTOP-J67FFRQ6:~/mnt$
```





VI. Conclusion

This project successfully designed and implemented a fully functional user-space inode-based file system using the FUSE framework. The implementation replicates the core architecture of classical UNIX file systems — superblock, bitmaps, inode table, and data blocks — and exposes it to the Linux VFS through a set of FUSE callbacks. All fundamental file operations including creation, reading, writing, directory management, deletion, and timestamp updates are operational and verified through standard Linux commands.

The FUSE framework proved to be an effective platform for file system experimentation, enabling rapid development and testing without kernel modification. The project provides a practical foundation for exploring advanced topics such as indirect block pointers for large file support, journaling for crash consistency, and performance benchmarking against native kernel file systems.

Future scope includes implementing single and double indirect block pointers to support larger files, adding a rename-across-directories operation, implementing file truncation with proper block reclamation, and conducting throughput benchmarks comparing the user-space FUSE overhead against ext4 on equivalent hardware.

References

[1] B. K. R. Vangoor, V. Tarasov, E. Zadok, “To FUSE or Not to FUSE: Performance of UserSpace-FileSystems”,USENIX_FAST_Conference,2017. https://www.usenix.org/system/files/conference/fast17/fast17vangoor.pdf

[2] A. Rajgarhia and A. Gehani, "Performance and Extension of User Space File Systems," in Proc. ACM Symposium on Applied Computing (SAC), 2010, pp. 206–213.